Kochbuch WISP

Wie WISP aus einer verrauschten Messreihe eine Differentialgleichung macht. Alle Schritte in einem Dokument, an drei echten Beispielen durchgerechnet, ohne Vorwissen: einem Lehrbuchfall, einem System mit zwei Gleichungen und einem Fall, in dem es schiefgeht. Bei jedem Schritt steht auch, warum er funktioniert.

WISP = **W**eak-form **I**ncremental **S**tructure **P**ursuit. Ersetzt die fünf Einzelkochbücher vom 27.10. (Weak Form, FROLS, FoBa, Trajektorien-Check, Residual-Wachstum).

Das Gericht in einem Absatz

WISP baut ein Modell Term für Term auf. Welcher Term als Nächstes dazukommt, entscheidet eine schnelle lineare Rechnung, die keine Ableitungen braucht (die *schwache Form*). Nach jedem neuen Term wird geprüft, ob ein alter Term dadurch überflüssig wurde, und der fliegt dann wieder raus. Jeder Zwischenstand ist ein Kandidat. Der Aufbau hört auf, wenn kein Term mehr hilft, als Rauschen es auch könnte. Dann wird jeder Kandidat einmal richtig an die Messdaten gefittet, simuliert, und ein Informationskriterium (BIC) wählt den besten.

Die Schritte auf einen Blick

1. **Zutaten bereitlegen:** Messreihe und Termkatalog.
2. **Schwache Form:** aus der Messreihe ein lineares Gleichungssystem machen.
3. **Wachsen (FROLS):** den Term nehmen, der am meisten vom Restfehler erklärt.
4. **Aufräumen (FoBa):** Terme entfernen, die überflüssig geworden sind.
5. **Aufhören:** wenn kein Term mehr stärker hilft als Rauschen.
6. **Fitten:** jeden Kandidaten einmal an die Messreihe anpassen, Start aus Schritt 2.
7. **Wählen (BIC):** den Kandidaten mit dem besten Verhältnis von Passung zu Größe nehmen.

8. **Abschmecken:** Struktur, Rekonstruktion und Generalisierung messen.

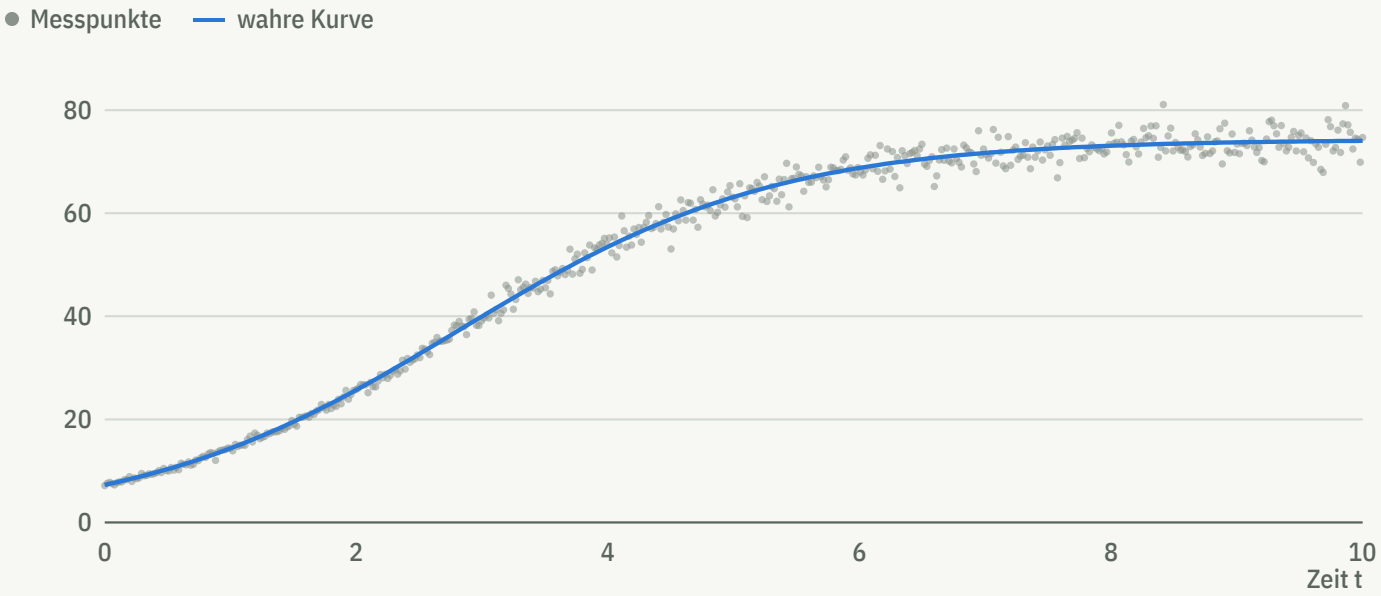
Das Beispiel, das durch alle Schritte führt

System 3, logistisches Wachstum (Strogatz S. 22): Eine Population wächst, bis sie an eine Kapazitätsgrenze stößt.

$$\dot{x} = 0{,}79 \cdot x \cdot (1 - x / 74{,}3) \quad = \quad 0{,}79 \cdot x \; - \; 0{,}0106 \cdot x^2$$

Die Wahrheit hat also zwei Terme: x und x^2. WISP weiß das nicht. Es bekommt nur 512 Messpunkte zwischen t = 0 und t = 10, Startwert 7,3, und auf jedem Punkt **3 % Rauschen** ($\sigma = 0{,}03$, multiplikativ). Alle Zahlen unten stammen aus dem echten Lauf dieser Zelle (`sys0003_ic1_s0.03_r0_n1`), nichts ist erfunden.

Die Messreihe: 512 Punkte mit 3 % Rauschen, dazu die wahre Kurve



Die Kurve steigt von 7,3 auf die Kapazitätsgrenze 74,3. Je größer x, desto größer das Rauschen (multiplikativ). Mit der Maus über die Grafik fahren zeigt die Werte.

Das Rezept

1 **Zutaten bereitlegen**

Die Messreihe: eine einzige Trajektorie x(t) mit einem Startwert. Eine zweite Trajektorie mit einem anderen Startwert gibt es auch, aber die bleibt im Kühlschrank: Sie ist nur für den Test am Ende (Generalisierung).

Der Termkatalog (die „Grammatik“): `1, u, u², u·v, u³, sin u, cos u` für jede Zustandsgröße. Mit einer Zustandsgröße sind das genau sechs Terme, mit zwei sind es 12, mit drei 19, mit vier 27.

IM BEISPIEL

Eine Zustandsgröße, also sechs mögliche Terme: `1, x, x², x³, sin x, cos x`.

2 Schwache Form: Messreihe in ein Gleichungssystem verwandeln

SINDy würde die Ableitung $\dot{x}$ aus den Daten schätzen. Bei Rauschen ist das gefährlich: Ableitungen verstärken Rauschen. Die schwache Form integriert stattdessen. Für jedes kurze Zeitfenster gilt immer:

```
x(Fensterende) − x(Fensteranfang)  =  ∫ rechte Seite dt  über das Fenster
```

Links steht die Differenz zweier Messwerte. Rechts steht für jeden Term sein Integral über das Fenster, mit der Trapezregel aus den Messpunkten berechnet. Fenster gibt es in fünf Längen (1, 2, 4, 8 und 16 Messpunkte), jedes Fenster ist eine Zeile. Am Ende steht ein lineares Gleichungssystem:

```
b  ≈  A · c       b = Differenzen,   A = Term-Integrale,   c = Koeffizienten
```

Für jede Auswahl von Termen löst man das per kleinster Quadrate in Millisekunden. Was übrig bleibt, ist der **Restfehler**, die Summe der quadrierten Reste (SSE).

IM BEISPIEL

512 Messpunkte ergeben 2.529 Fenster-Zeilen. Ohne jeden Term ist der Restfehler 20.320.

> WARUM FUNKTIONIERT DAS?
>
> Rechnen wir es im Beispiel nach. Bei $x \approx 50$ ist das Rauschen 3 % von 50, also eine Standardabweichung von etwa 1,5. Die Messpunkte liegen $\Delta t = 0{,}0196$ auseinander.

- **Ableitung schätzen** (Differenz zweier Nachbarpunkte durch Δt): Das Rauschen wird durch das winzige Δt geteilt und landet bei etwa $1{,}5 \cdot \sqrt{2} / 0{,}0196 \approx$ **108**. Die wahre Ableitung ist aber höchstens 14,7. Das Rauschen ist **siebenmal größer als das Signal**.
- **Schwache Form, Fenster mit 16 Punkten** (0,31 Zeiteinheiten): Links steht x(Ende) − x(Anfang). Das Rauschen davon bleibt bei $1{,}5 \cdot \sqrt{2} \approx 2{,}1$, denn es wird durch nichts geteilt. Das Signal ist die Änderung über das Fenster, bis zu $14{,}7 \cdot 0{,}31 \approx 4{,}6$. Das Rauschen ist jetzt **kleiner als das Signal**. Rechts, in den Integralen der Terme, mittelt die Trapezregel das Rauschen über 17 Punkte sogar weiter herunter.

Der Trick ist der Hauptsatz der Analysis: Die Änderung von x über ein Intervall ist das Integral der Ableitung. Man muss die Ableitung also nie kennen. Die verschiedenen Fensterlängen kombinieren beides: Kurze Fenster sehen schnelle Änderungen, lange Fenster haben besseres Signal-Rausch-Verhältnis.

Ein Haken bleibt: Das Rauschen steckt auch *in* den Integralen rechts (man integriert ja verrauschte x-Werte). Dadurch sind die schwachen Koeffizienten leicht verzerrt, bei x^2 und x^3 stärker als bei x. Deshalb wird in Schritt 6 noch einmal richtig gefittet.

Darum ist WISP so billig im Suchen: Koeffizienten werden ausgerechnet, nicht durch Simulieren gesucht. Bei unregelmäßig abgetasteten Daten ($\rho = 0{,}5$) läuft dasselbe auf den tatsächlichen Messzeiten.

3 **Wachsen: den Term nehmen, der am meisten erklärt (FROLS)**

Für jeden noch nicht benutzten Term: probeweise hinzufügen, **alle** Koeffizienten neu ausrechnen, schauen, wie stark der Restfehler sinkt. Der Term mit dem größten Gewinn kommt dazu. Bei mehreren Gleichungen wird jeder Gewinn durch die Größe der jeweiligen linken Seite geteilt, damit keine Gleichung alle Schritte an sich zieht; pro Schritt kommt genau ein Term in genau eine Gleichung.

IM BEISPIEL, SCHRITT 1 UND 2

Schritt 1: Der größte Gewinn kommt von der Konstante `1` (15 % des Restfehlers). Falsch! Die Wahrheit hat keine Konstante. Aber am Anfang ist eine Konstante die beste Ein-Term-Näherung an eine Kurve, die von 7 auf 74 steigt.

Schritt 2: Als Nächstes `x³` (6 %). Auch falsch. Modell jetzt: `1, x³`.

WARUM „ALLE KOEFFIZIENTEN NEU AUSRECHNEN“?

Auf einer einzigen Trajektorie sehen sich die Terme sehr ähnlich: Wenn x wächst, wachsen x^2 und x^3 mit. Im Beispiel senkt jeder Term für sich allein den Restfehler von anfangs 20.320 um:

Term allein	Gewinn
`1`	`3.060`
`x`	`1.600`
`x²`	`897`
`x³`	`583`
`cos x`	`225`
`sin x`	`5`

Diese Zahlen überlappen stark: Was x erklärt, erklärt x^2 zum großen Teil auch. Würde man einfach die drei Besten nehmen, bekäme man dreimal fast dieselbe Information. FROLS fragt deshalb nach jedem Schritt neu: *Was erklärt dieser Term zusätzlich zu dem, was schon drin ist?* Technisch heißt das, der neue Term wird gegen die vorhandenen „orthogonalisiert“, also nur sein neuer Anteil zählt. Die naive Variante, die nur nach Korrelation mit dem Rest auswählt (OMP), fand in unseren Tests auf dim 2 die fehlenden Terme in 4 von 22 Fällen, FROLS in 19 von 22.

Warum dann trotzdem zuerst die falsche Konstante? Weil FROLS gierig ist: Es nimmt immer den besten nächsten Schritt, ohne vorauszuschauen. Ein einzelner Term, der die ganze Kurve grob trifft, gewinnt gegen zwei Terme, die nur zusammen perfekt passen. Diesen Fehler repariert Schritt 4.

Das ist typisch, gerade unter Rauschen: Frühe Schritte nehmen Terme, die grob passen, nicht die richtigen. Darum braucht es Schritt 4. Das Verfahren heißt FROLS

(Billings, Chen & Korenberg 1989).

4 Aufräumen: Überflüssiges wieder entfernen (FoBa)

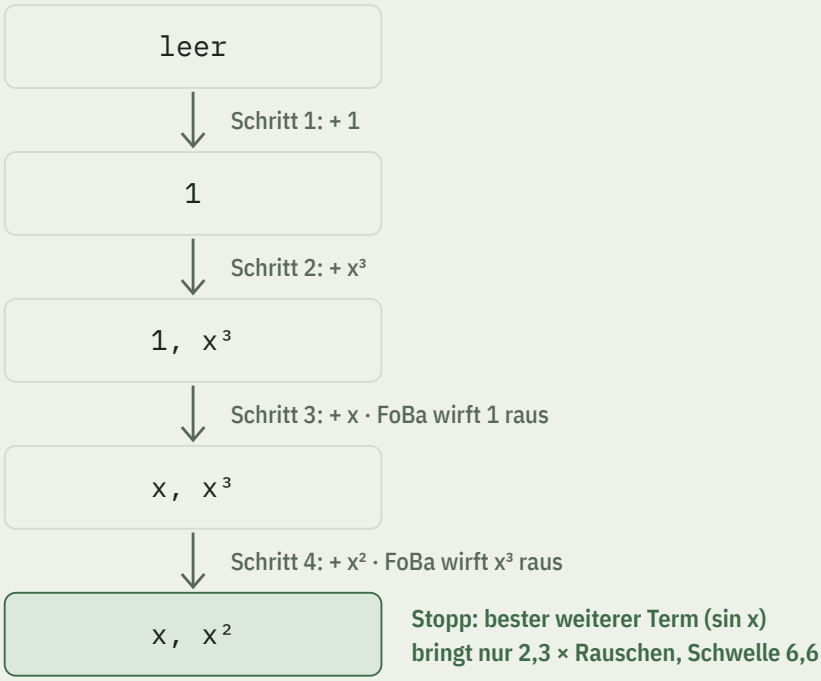
Nach jedem neuen Term wird jeder alte Term geprüft: Wie stark stiege der Restfehler, wenn man ihn wegließe? Ist das weniger als die Hälfte dessen, was der neue Term gerade gebracht hat, fliegt er raus. Wiederholen, bis nichts mehr rausfliegt.

IM BEISPIEL, SCHRITT 3 UND 4

Schritt 3: x kommt dazu. Jetzt ist die Konstante überflüssig und **fliegt raus**. Modell: $x,\ x^3$.

Schritt 4: x^2 kommt dazu. Jetzt ist x^3 überflüssig und **fliegt raus**. Modell: $x,\ x^2$, genau die Wahrheit.

Der ganze Pfad im Beispiel



Jeder Kasten ist ein Kandidat. Grün: die Wahrheit, bei der die Q7-Regel stoppt.

WARUM GERADE „WENIGER ALS DIE HÄLFTE“?

Die Regel kommt aus dem FoBa-Algorithmus von Tong Zhang (2011) und hat einen eleganten Grund. Jeder Vorwärtsschritt senkt den Restfehler um einen Gewinn G. Ein Rückwärtsschritt darf ihn wieder erhöhen, aber nur um weniger als G/2. Damit gilt: Nach jedem Paar aus Hinzufügen und Entfernen ist der Restfehler um mindestens G/2 kleiner als vorher. Der Fehler sinkt also

> immer, das Verfahren kann nicht im Kreis laufen und endet garantiert. Zhang hat außerdem bewiesen, dass FoBa unter bestimmten Bedingungen die wahre Struktur findet, die reine Vorwärtssuche dagegen nicht.
>
> Im Beispiel: In Schritt 3 bringt x einen Gewinn von 1,8 % des Gesamtfehlers. Die Konstante zu entfernen kostet weniger als 0,9 %, also fliegt sie. Mit x im Modell war sie nur noch ein schlechter Ersatz für etwas, das x besser kann.

> Das ist FoBa (Zhang 2011); der Faktor ½ ist der veröffentlichte Standardwert, nicht von uns eingestellt. Genau diesen Rückwärtsschritt hatte EvoGrow nicht: Dort konnte ein falscher Term nie wieder raus.

5 Aufhören: wenn kein Term mehr hilft, als Rauschen es auch könnte

Jeder Zwischenstand ist ein Kandidat. Aber wann hört man auf? Unter Rauschen sinkt der Restfehler nie auf null, irgendein Term „hilft“ immer ein bisschen, weil er Rauschen nachzeichnet. Die Regel (Q7):

```
Für jeden freien Term:  Gewinn / erwarteter Gewinn eines
Rauschterms
Bei keinem Term größer als 6,6  →  Gleichung hört auf.
```

Den „erwarteten Gewinn eines Rauschterms“ rechnen wir aus dem Rauschen aus, das wir vorher aus den Daten selbst schätzen: An jedem Punkt wird ein kleines kubisches Polynom durch die Nachbarpunkte gelegt; wie stark die Messwerte darum streuen, ist das Rauschen. Die 6,6 ist die übliche Signifikanzschwelle (1 %, χ^2-Test), nicht eingestellt.

IM BEISPIEL

Modell	größtes Verhältnis Gewinn / Rauschgewinn	weiter?
leer	9.310	ja
1	2.380	ja
1, x^3	935	ja
x, x^3	305	ja
x, x^2	2,3 (bei sin x)	**nein, Stopp**

Bei der Wahrheit angekommen, würde der beste weitere Term (`sin x`) den Fehler nur 2,3-mal so stark senken wie ein Rauschterm. Das ist nicht signifikant: Stopp. Die Kandidatenliste hat vier Einträge.

WARUM IST DAS DIE RICHTIGE FRAGE, UND WARUM 6,6?

Stell dir vor, ein Term hat in Wahrheit nichts mit dem System zu tun. Hinzufügen senkt den Restfehler trotzdem etwas, denn er passt sich zufällig ein bisschen an das Rauschen an. Wie viel im Mittel, kann man ausrechnen: Man kennt das Rauschen an jedem Punkt (geschätzt) und weiß, wie es in jede Fensterzeile eingeht. Daraus folgt der „erwartete Gewinn eines Rauschterms“.

Teilt man den echten Gewinn durch diesen Erwartungswert, bekommt man bei einem nutzlosen Term eine Zahl, die bei Gauß-Rauschen einer bekannten Verteilung folgt (χ^2 mit einem Freiheitsgrad): im Mittel 1, in 99 % der Fälle unter 6,6. Ein Wert von 9.310 ist also eindeutig echt, ein Wert von 2,3 ist ganz normales Rauschen.

Warum Q6 scheiterte und das hier nicht: Q6 fragte „Ist der Restfehler schon so klein wie das Rauschen?“. Der Restfehler ist aber eine riesige Summe über Tausende Fensterzeilen und schwankt allein durch das Rauschen stark. Ein kleiner wahrer Term ändert diese Summe weniger, als sie ohnehin schwankt, und ging darin unter. Der Gewinn eines einzelnen Terms ist dagegen eine einzelne, gezielte Messung, und dort fällt ein echter Term auf.

Ehrliche Einschränkungen: Wir prüfen mehrere Terme gleichzeitig. Bei vier Kandidaten ist die Chance, dass irgendein nutzloser Term zufällig über 6,6 liegt, etwa 4 % statt 1 %. Und ein wahrer Term, dessen Beitrag kleiner ist als das Rauschen, bleibt unsichtbar. Das ist keine Schwäche der Regel, sondern eine Grenze dessen, was in den Daten steckt.

Welcher Term als Nächstes tatsächlich dazukommt, entscheidet weiter FROLS (Schritt 3); der Test fragt nur, ob überhaupt noch einer signifikant hilft. Vorher stand hier eine feste Grenze von 6 Termen pro Gleichung. Die war aus der Lösung abgeleitet (doppelt so viele wie die größte wahre Gleichung) und ist damit methodisch unsauber. Mehr dazu unten unter „Was wir gelernt haben“.

6 Fitten: jeden Kandidaten einmal richtig anpassen

Die Koeffizienten aus der schwachen Form sind nur eine Näherung. Jetzt wird jeder Kandidat **simuliert** und an die Messreihe angepasst: Optimierer BFGS (derselbe wie in EvoGrow), Start bei den schwachen Koeffizienten, **genau ein Versuch**, Koeffizienten auf [−10, 10] begrenzt. Simuliert wird ab dem ersten Messpunkt.

```
MSE = Mittel über alle Messpunkte von (x_simuliert − x_gemessen)²
```

IM BEISPIEL, KANDIDAT 4 (DIE WAHRHEIT)

Schwacher Start `0,785·x − 0,0105·x²`, nach dem Fit `0,795·x − 0,0107·x²`. Wahr ist `0,79·x − 0,0106·x²`.

WARUM NOCH EINMAL FITTEN, WENN DIE SCHWACHE FORM SCHON KOEFFIZIENTEN LIEFERT?

Zwei Gründe. Erstens sind die schwachen Koeffizienten unter Rauschen leicht verzerrt (siehe Schritt 2: Rauschen steckt auch in den Integralen). Zweitens zählt am Ende, ob das Modell als Ganzes die Kurve nachbildet, nicht ob es viele kurze Fenster einzeln gut trifft. Kleine Fehler in der rechten Seite summieren sich beim Simulieren über die Zeit auf. Der Fit im Trajektorienraum korrigiert genau das.

Warum dann nicht gleich nur im Trajektorienraum fitten, wie EvoGrow? Weil dieses Optimierungsproblem viele Täler hat. Von einem zufälligen Start landet BFGS oft im falschen Tal. Der schwache Start liegt schon im richtigen Tal, deshalb reicht ein einziger Versuch.

Eine bekannte Schwäche: Der Fit startet die Simulation am ersten Messpunkt, und der ist verrauscht. Das zweite Beispiel unten zeigt, was das anrichten kann.

Ein einziger Fit vom schwachen Start schlägt zehn Zufallsstarts (Q3). Fast die ganze Rechenzeit von WISP steckt in diesem Schritt; deshalb ist eine kurze

Kandidatenliste so wertvoll.

7 Wählen: das kleinste BIC gewinnt

```
BIC = N · log(MSE) + k · log(N)      N = Messpunkte × Dimension,
k = Zahl der Terme
```

Der erste Teil belohnt gute Passung, der zweite bestraft jeden Term. Kleiner ist besser. Keine Stellschraube.

IM BEISPIEL

Kandidat	Terme	MSE	BIC
1	1	141,1	2.540
2	1, x^3	29,8	1.751
3	x, x^3	5,31	868
4	x, x^2	3,04	**581**

Die Wahrheit gewinnt deutlich. Zum Vergleich: Mit der alten 6-Term-Grenze lief der Pfad weiter bis zu Kandidat 8 mit allen sechs Termen. Der beste davon (x, x^2, x^3, sin x) erreichte BIC 598, nur 17 Punkte hinter der Wahrheit. Hier hat BIC trotzdem richtig gewählt; die neue Regel erspart vier Fits.

WAS IST DAS BIC EIGENTLICH?

Das BIC (Schwarz 1978) beantwortet die Frage: Welches Modell macht die Daten am wahrscheinlichsten, wenn man fair berücksichtigt, dass mehr Parameter immer besser passen? Der erste Teil, N · log(MSE), ist bis auf eine Konstante „minus zweimal die Log-Likelihood“ bei Gauß-Fehlern, also ein Maß für die Passung. Der zweite Teil, k · log(N), ist der Preis für jeden Parameter. Er kommt nicht aus einer Faustregel, sondern aus einer Näherung: Man mittelt die Passung über alle plausiblen Parameterwerte, und jeder zusätzliche Parameter „verdünnt“ diese Mittelung um etwa log(N).

Im Beispiel: Von Kandidat 3 zu 4 sinkt der MSE von 5,31 auf 3,04. Das ist N · log(5,31/3,04) = 512 · 0,56 ≈ 285 Punkte Verbesserung, bei gleicher Termzahl. Ein zusätzlicher Term würde 6,2 Punkte kosten (log 512). Er lohnt sich also nur, wenn er den MSE um mehr als etwa 1,2 % senkt.

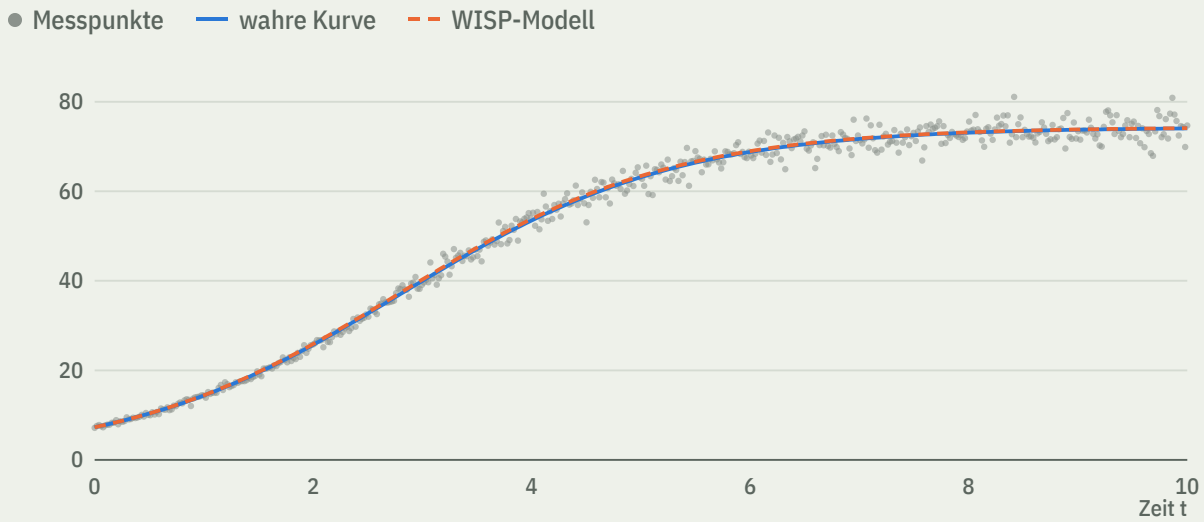
> Bekannte Schwäche: Das BIC nimmt an, dass die Fehler an den 512 Punkten unabhängig sind. Bei einer simulierten Kurve stimmt das nicht. Liegt das Modell bei t daneben, liegt es bei t + Δt fast genauso daneben. Das BIC überschätzt dadurch, wie viel Information in den Daten steckt, und neigt unter Rauschen zu etwas zu großen Modellen.

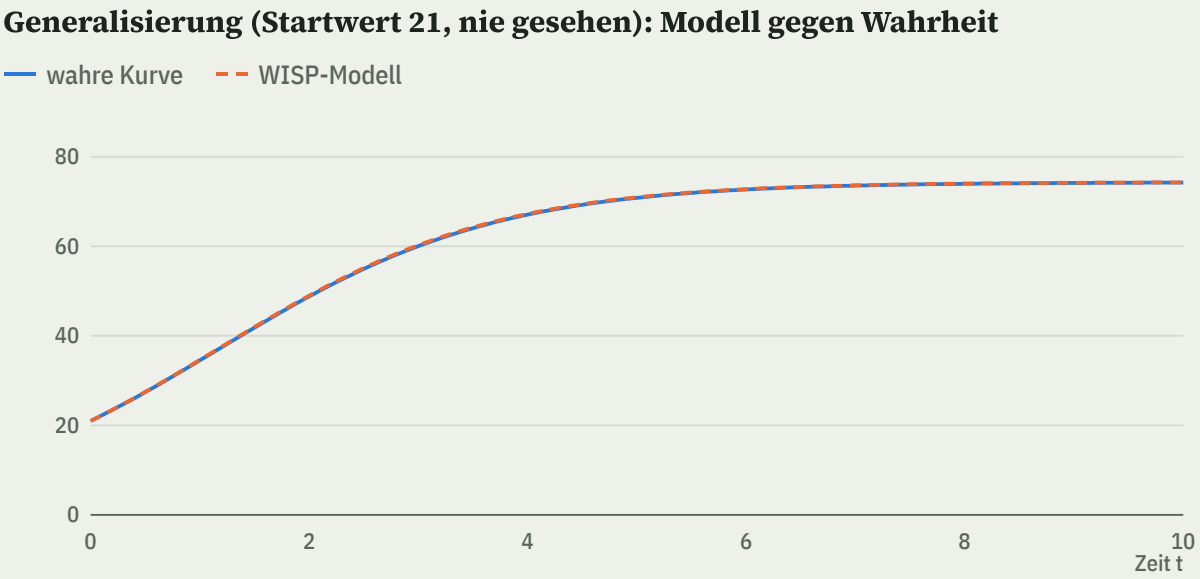
> Warum die Simulation nötig ist: Wählt man die Größe nur im schwachen Raum, nimmt das Verfahren im Median vier Terme zu viel (Q4). Ein kleiner Fehler in $\dot{x}$ bleibt in der schwachen Form klein, summiert sich beim Simulieren aber über die ganze Kurve.

8 Abschmecken: was gemessen wird

- **Struktur:** Stimmen die gewählten Terme mit der Wahrheit überein? Erst werden winzige Koeffizienten weggeschnitten (kleiner als ein Tausendstel des größten), dann wird verglichen.
- **Rekonstruktion:** Modell von der sauberen Trainings-Startbedingung simulieren, R^2 gegen die saubere Kurve. Treffer, wenn $R^2 > 0{,}9$.
- **Generalisierung:** Modell von der **anderen** Startbedingung aus simulieren, die nie gesehen wurde. Treffer, wenn $R^2 > 0{,}9$. Das ist der eigentliche Test, ob das Modell den Mechanismus verstanden hat.

Training (Startwert 7,3): gewähltes Modell gegen die Wahrheit





IM BEISPIEL

Struktur exakt richtig. Rekonstruktion $R^2 = 1{,}0000$, Generalisierung $R^2 = 1{,}0000$. Ein Lehrbuchfall.

> Immer beide Metriken zusammen: Ein strukturell falsches Modell kann trotzdem eine perfekte Rekonstruktion haben (Kandidat 3 oben: $R^2 = 0{,}996$ mit dem falschen Term x^3).

Zweites Beispiel: Räuber und Beute, zwei Gleichungen

System 27, Lotka-Volterra. Beute x vermehrt sich und wird gefressen, Räuber y stirbt ohne Beute und lebt vom Fressen:

```
ẋ =  1,84 · x − 1,45 · x·y
ẏ = −3,00 · y + 1,62 · x·y
```

Wahr sind also zwei Terme pro Gleichung. Der Katalog hat bei zwei Zustandsgrößen zwölf Terme pro Gleichung, also 24 Möglichkeiten insgesamt. Daten: 512 Punkte, 3 % Rauschen, Start (8,3; 3,4). Zelle `sys0027_ic1_s0.03_r0_n1`.

Neu bei zwei Gleichungen: die Gleichungen konkurrieren

Pro Schritt kommt genau ein Term in genau eine Gleichung. Welche, entscheidet der Gewinn, geteilt durch die Größe der jeweiligen linken Seite, damit beide Gleichungen fair verglichen werden:

Schritt	Gleichung	neuer Term	Gewinn (Anteil)	Modell danach
1	$\dot{x}$	x·y	67 %	ẋ: x·y | ẏ: –
2	$\dot{y}$	x·y	40 %	ẋ: x·y | ẏ: x·y
3	$\dot{y}$	y	59 %	ẋ: x·y | ẏ: y, x·y
4	$\dot{x}$	x	31 %	ẋ: x, x·y | ẏ: y, x·y

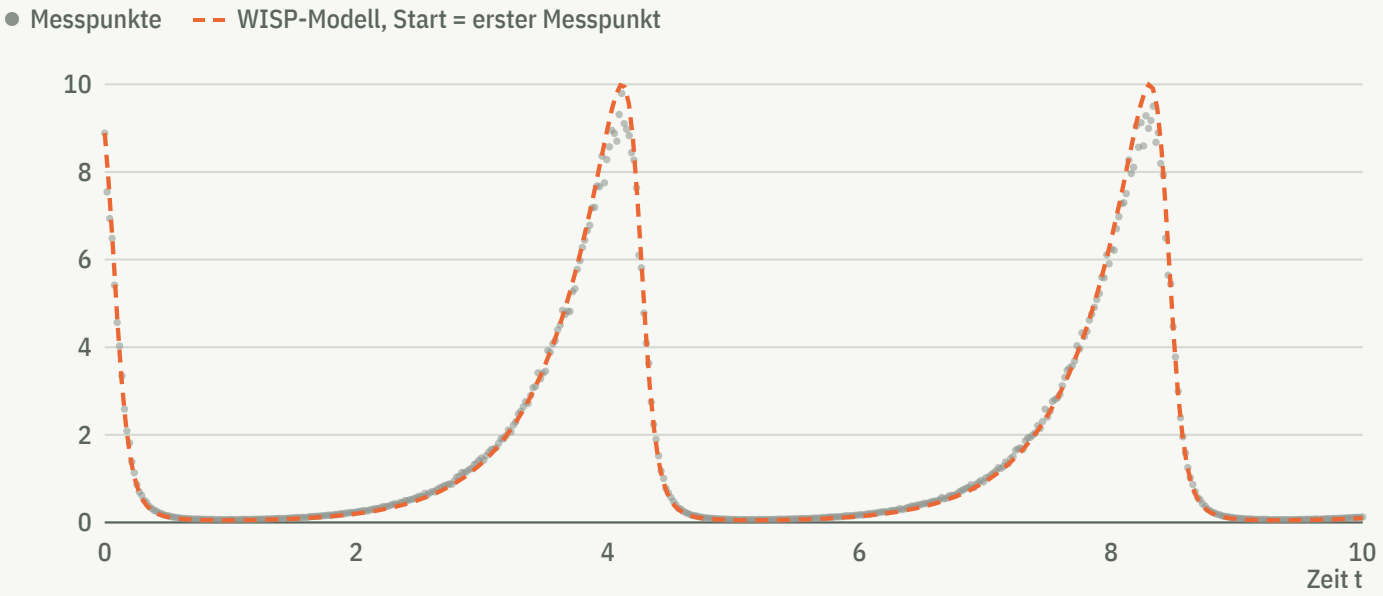
Kein falscher Term, nichts musste entfernt werden. Danach stoppen beide Gleichungen: Der beste weitere Term bringt in $\dot{x}$ nur 3,5-mal und in $\dot{y}$ nur 4,0-mal so viel wie ein Rauschterm, beides unter 6,6. Vier Kandidaten, BIC wählt den vierten mit großem Abstand (−3.782 gegen +1.866 für den dritten). **Struktur exakt richtig.** Die gefitteten Koeffizienten:

```
ẋ =  1,96 · x − 1,52 · x·y        (wahr  1,84 / −1,45)
ẏ = −2,92 · y + 1,56 · x·y        (wahr −3,00 /  1,62)
```

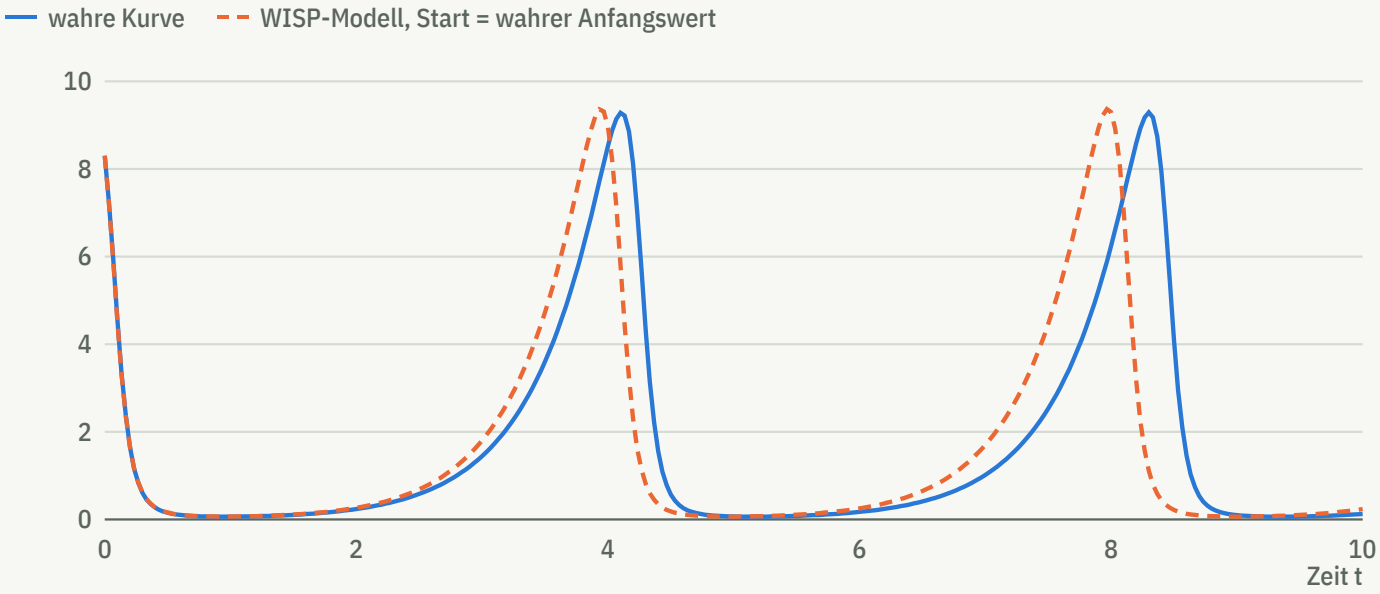
Und trotzdem nur $R^2 = 0{,}52$ bei der Rekonstruktion

Die Struktur stimmt, die Koeffizienten liegen nur um etwa 5 % daneben, und doch erreicht das Modell bei der Rekonstruktion nur $R^2 = 0{,}52$, bei der Generalisierung 0,82. Was ist passiert?

Fit auf die Trainingsdaten: Beute x, Modell ab dem verrauschten ersten Messpunkt



Dasselbe Modell, gestartet am wahren Anfangswert (8,3; 3,4): Beute x gegen die Wahrheit



Oben: Auf den Trainingsdaten passt das Modell sehr gut, gestartet am **verrauschten ersten Messpunkt** (8,88; 3,48). Der wahre Start ist (8,3; 3,4). Unten: Startet man dasselbe Modell am wahren Anfangswert, läuft es mit der Zeit aus dem Takt. Der Grund liegt im System: Lotka-Volterra schwingt auf geschlossenen Bahnen, und welche Bahn mit welcher Periode, hängt vom Startpunkt ab. Der Fit hat die Koeffizienten so verbogen, dass das Modell vom *falschen* Start aus die Daten trifft. Vom *richtigen* Start aus stimmt dann die Periode nicht mehr, und der kleine Unterschied wächst Schwingung für Schwingung.

Die Zahl dazu: Mit denselben Koeffizienten ist der Trainingsfehler vom verrauschten Start 0,024, vom wahren Start 2,96, also mehr als hundertmal so groß.

WAS MAN DARAUS LERNT

Erstens: Struktur und R^2 messen verschiedene Dinge. Hier ist die Struktur perfekt und das R^2 schlecht. Darum berichten wir immer beide. Zweitens: Bei schwingenden Systemen ist der Startwert so wichtig wie die Koeffizienten. Genau deshalb soll die Paper-Version den Startwert mitschätzen (Variante C), statt dem ersten verrauschten Messpunkt zu vertrauen.

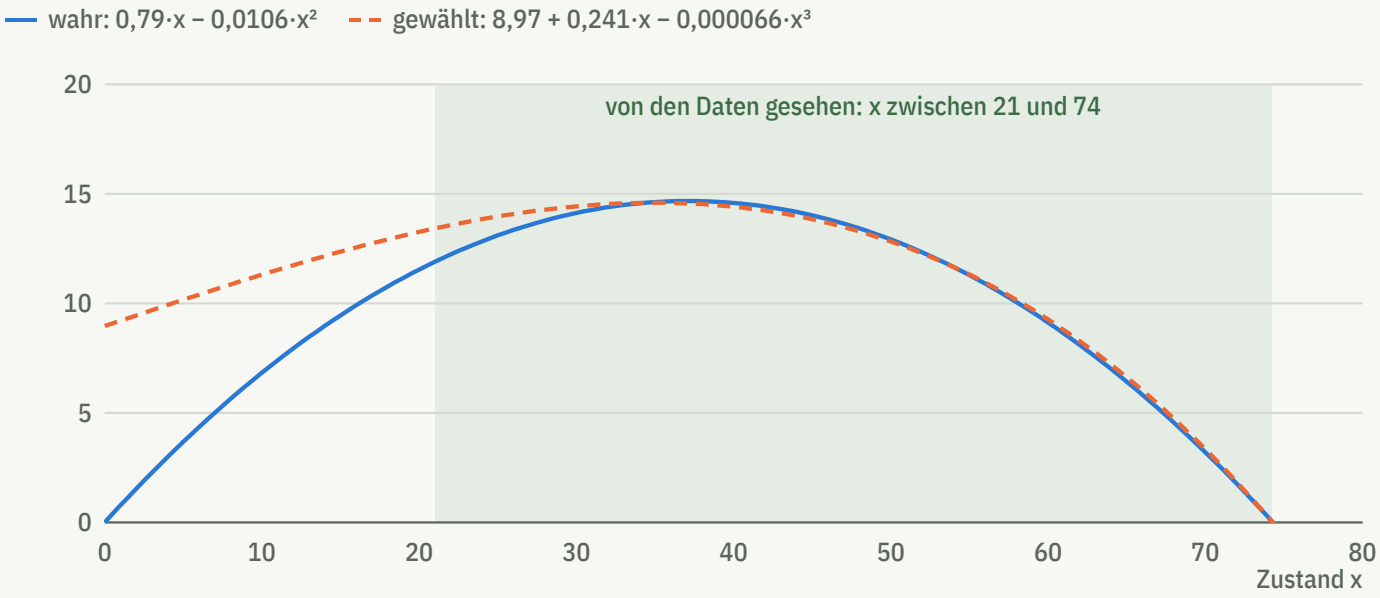
Drittes Beispiel: wenn es schiefgeht

Dieselbe logistische Gleichung wie im ersten Beispiel, dasselbe Rauschen von 3 %. Einziger Unterschied: Die Messreihe startet bei **21** statt bei 7,3. Zelle `sys0003_ic2_s0.03_r0_n1`.

Schritt	neuer Term	Modell danach	bester weiterer Term, Verhältnis zu Rauschen
1	1	1	7.210
2	x^3	1, x^3	50
3	x	1, x, x^3	**0,03 (x^2) → Stopp**

Diesmal fliegt nichts raus. Als x dazukommt, bringt das Entfernen der Konstante zu viel Verlust, sie bleibt. Und dann bringt x^2 nichts mehr, nicht einmal so viel wie ein Rauschterm. **Die Wahrheit x, x^2 kommt nie auf die Liste.** BIC wählt `1, x, x³`; nach dem Wegschneiden des winzigen x^3-Koeffizienten bleibt `8,97 + 0,241·x`. Rekonstruktion R^2 = 0,9996, Generalisierung vom Startwert 7,3 aus R^2 = 0,964. Beide über 0,9, die Struktur aber falsch.

Die rechte Seite $\dot{x} = f(x)$: Wahrheit gegen das gewählte falsche Modell



Warum? Die Grafik zeigt die rechte Seite, also wie schnell x sich bei jedem Wert von x ändert. Im grünen Bereich, den die Daten tatsächlich gesehen haben (x zwischen 21 und 74), liegen die beiden Kurven fast übereinander; der größte Unterschied ist 1,5 bei Werten bis 15. Erst unterhalb von 20, wo es keine Daten gibt, laufen sie auseinander: Bei x = 7,3 sagt die Wahrheit 5,2, das falsche Modell 10,7.

Die Daten enthalten die Information schlicht nicht. Eine Kurve, die bei 21 startet, sieht nur die obere Hälfte des logistischen Wachstums, und auf diesem Stück kann eine andere Formel das Verhalten genauso gut beschreiben. Im ersten Beispiel startete die Kurve bei 7,3 und zeigte den ganzen S-Bogen. Dort war x^2 eindeutig nötig.

WAS MAN DARAUS LERNT

- **Kein Verfahren kann finden, was die Daten nicht zeigen.** Das gilt für WISP, für WSINDy und für jedes andere. Unter Rauschen wird der gesehene Bereich effektiv noch kleiner, weil feine Unterschiede im Rauschen untergehen.
- **R^2 über 0,9 heißt nicht „richtig".** Das falsche Modell generalisiert hier sogar ordentlich, weil der Startwert 7,3 nicht allzu weit vom gesehenen Bereich entfernt ist. Weiter weg würde es scheitern.
- Darum steht unter Rauschen die exakte Wahrheit nur in etwa der Hälfte der Fälle auf der Liste: Bei der logistischen Gleichung schafft es die Messreihe ab 7,3 in den meisten Fällen, die ab 21 fast nie.

Was wir bisher gemessen haben

Ohne Rauschen (Q4)

Struktur exakt: dim 1 82 %, dim 2 90 %, dim 3 fittbar 3 von 4. EvoGrow C-1 auf denselben Einheiten: 47 %, 37 %, 0 von 12. Bei etwa 4 % der Rechenkosten von EvoGrow.

Mit Rauschen gegen WSINDy (Q5-W, vorläufig)

Gegner ist das veröffentlichte WSINDy (Messenger & Bortz), portiert aus dem Original-MATLAB-Code. Der Abgleich gegen MATLAB steht noch aus, deshalb sind alle Zahlen vorläufig. Gezählt sind Zellen mit Treffer.

verrauscht, gleichmäßig abgetastet	Struktur: WISP	WSINDy	Generalisierung: WISP	WSINDy
dim 1 (352 Zellen)	111	96	**228**	158
dim 2 (320 Zellen)	83	84	**199**	124

verrauscht, unregelmäßig abgetastet	Struktur: WISP	WSINDy*	Generalisierung: WISP	WSINDy*
dim 1 (396 Zellen)	126	125	**271**	204
dim 2 (360 Zellen)	**109**	68	**222**	115

* WSINDy braucht gleichmäßige Zeitabstände; hier bekommt es die Daten linear auf ein gleichmäßiges Raster interpoliert, die stärkere seiner beiden Varianten. Diese Zahlen stammen noch aus dem Lauf mit der 6-Term-Grenze.

Lesart: WISP generalisiert unter Rauschen deutlich besser als WSINDy; bei der exakten Struktur liegt es gleichauf oder vorn. Absolut trifft unter Rauschen auch WISP die exakte Struktur nur in etwa einem Viertel bis einem Drittel der Zellen. Das ist ein offenes Problem des ganzen Feldes, kein uns bekanntes erklärbares Verfahren ist da besser.

Was wir über das Aufhören gelernt haben

1 **Feste Grenze 6 (bis Q5)**
Funktioniert, ist aber aus der Lösung abgeleitet. Und sie kappt auf dim 2 bis 4 oft, bevor alle wahren Terme auf der Liste sind.

2 **„Fehler so klein wie das Rauschen“ (Q6): durchgefallen**
Hört gleichzeitig zu früh und zu spät auf: In 591 von 2.788 Gleichungen wurde ein wahrer Term abgeschnitten. Grund: Ein kleiner wahrer Term ändert die *Höhe* des Fehlers weniger, als diese ohnehin schwankt. Nützliches Nebenergebnis: Die Rauschschätzung selbst ist sehr gut (geschätzt durch wahr im Median 0,96–1,03).

3 **„Hilft der nächste Term mehr als Rauschen?“ (Q7): vielversprechend**
Schaut auf den *Gewinn* statt auf die Höhe. Ohne jedes Wissen über die Lösung steht die exakte Wahrheit genauso oft auf der Liste wie mit der Grenze 6 (verrauscht: dim 1 49 / 50 %, dim 2 69 / 71 %, dim 3 54 / 53 %), bei halb so langer Liste. Ob das Endergebnis gleich gut bleibt, zeigt der Fit-Lauf, der gerade startet (3.070 neue Fits, ca. 1.100 Kernstunden). Vorher festgelegt: Q7 ersetzt die 6, wenn es höchstens 5 Prozentpunkte schlechter ist.

Woher die Bausteine kommen, und was neu ist

Baustein	Herkunft
Katalog von Termen, sparsames Modell daraus	SINDy, Brunton, Proctor & Kutz 2016
Schwache Form statt Ableitungen	WSINDy, Messenger & Bortz 2021; Integralform schon früher (Schaeffer & McCalla 2017)
Term für Term mit Neuausrechnen	FROLS, Billings, Chen & Korenberg 1989 (NARX-Systemidentifikation)
Vorwärts mit Rückwärts-Prüfung	FoBa, Zhang 2011
Kandidaten simulieren, Auswahl per Informationskriterium	Mangan, Kutz, Brunton & Proctor 2017
Signifikanztest für den nächsten Term	klassische schrittweise Regression (F-Test), hier mit der Rauschfortpflanzung in den schwachen Raum wie bei WENDy (Bortz, Messenger & Dukic 2023)

Kein einzelner Baustein ist neu. Neu ist ihre Kombination, und jeder Teil hat einen klaren Grund: die schwache Form, damit die Suche schnell und rauschfest ist; das gierige Wachsen mit Rückwärtsprüfung, damit nur eine kurze Liste entsteht und falsche Terme wieder gehen können; die Simulation mit Refit, weil am Ende das Verhalten über die Zeit zählt. Der nächste Verwandte, WSINDy, sucht stattdessen über eine Schwelle auf allen Koeffizienten gleichzeitig und wählt die Schwelle im schwachen Raum. Mangan 2017 simuliert zwar die Kandidaten, aber ohne nichtlinearen Refit. Ein Gutachter würde fragen: „Ist das nicht Mangan 2017 mit schwacher Form?“ Die Antwort muss eine Ablation sein, die zeigt, welcher Teil wie viel beiträgt. Die steht noch aus.

Was noch offen ist

VOR DEM PAPER

- **MATLAB-Abgleich:** Unser WSINDy-Port muss gegen das Original geprüft werden (Lizenz kommt).
- **Q7-Fits:** laufen als Nächstes auf Orion.
- **Startwert mitschätzen (Variante C):** Heute startet jede Simulation vom ersten, verrauschten Messpunkt. Für die Paper-Version wird der Startwert mitgefittet. Erst auf einer Teilmenge.
- **Warum die Wahrheit unter Rauschen oft nicht auf der Liste steht:** In etwa der Hälfte der verrauschten Zellen fehlt sie. Das ist die eigentliche Baustelle für eine spätere Version.
- **Einfachheit:** Jeder Baustein muss in einem Satz erklärbar bleiben. Scheitert etwas, nehmen wir lieber eine offen benannte Einschränkung in Kauf, als Reparaturen zu stapeln.

WISP in einem Satz pro Baustein

Schwache Form	Statt Ableitungen zu schätzen, vergleicht man Differenzen von Messwerten mit Integralen der Terme.
FROLS	Nimm den Term, der nach gemeinsamem Neuausrechnen den Restfehler am stärksten senkt.
FoBa	Wirf einen alten Term raus, wenn er weniger als halb so viel bringt wie der neue.
Stopp (Q7)	Ein Term kommt nur dazu, wenn er mehr bringt, als Rauschen plausibel könnte.
Fit	Ein BFGS-Fit pro Kandidat, gestartet bei den schwachen Koeffizienten.
BIC	Passung minus Strafe pro Term, das kleinste gewinnt.

Kleines Glossar

Restfehler (SSE)	Wie viel der Daten das aktuelle Modell noch nicht erklärt, als eine Zahl.
Kandidat	Ein Zwischenstand auf dem Pfad, also ein mögliches Modell.
Pfad	Die Folge aller Kandidaten, vom leeren Modell bis zum Stopp.
σ, ρ	Rauschstärke (bis 5 %) und Anteil weggelassener Messpunkte (0 oder 50 %).
dim	Zahl der Zustandsgrößen: dim 1 eine Gleichung, dim 2 zwei gekoppelte usw.
Stratum	Gruppe, die nie mit anderen gemittelt wird (dim 1, dim 2, dim 3 fittbar, dim 4).
WSINDy	Der wichtigste Gegner: SINDy in schwacher Form, mit automatischer Schwellenwahl.

Zum Weiterlesen im Repo

- `docs/sideproject_residual_growth_protocol*.md`: alle eingefrorenen Protokolle (Q1–Q7, Q5-W)
- `DIARY.md`, Einträge 17.10. bis 31.10.: alle Zahlen
- `analysis/scripts/aggregate/build_sideproject_q7_paths.py`: Schritte 2–5 im Code
- `studies/regression/sideproject_q5_noise_fits.jl`: Schritte 6–8 im Code